---------- NumPy ----------

NumPy (Numerical Python) is an open-source Python library used for high-performance numerical computing. It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these data structures efficiently

1. Creating Arrays
2. numpy array attributes
3. Array initialization methods
4. Array Indexing and Slicing
5. Array Reshaping and Flattening
6. Array Stacking and Splitting
7. Mathematical Operatoins on Array
8. Mathematical Operatoins on Multiple Array
9. Statistical Functions
10. Array Comparison
11. Broadcasting
12. Handling with nan & inf (Dealing with Missing Values)
13. Views vs Copies
14. Transpose of a Matrix
15. Conditional-Based Operations on Array
16. Save & Load Arrays

In [ ]:

In [147…
```
# Install Numpy
!pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\satya\anaconda3\lib\site-package
s (2.1.3)
```

In [3]:
```
# Import Numpy Library
import numpy as np
```

In [149…
```
# Numpy version installed in the system
print(np.__version__)
```

```
2.1.3
```

1. Creating Arrays

In [151…
```
list1=[1,2,3,4]
print(list1)
```

```
[1, 2, 3, 4]
```

In [152…
```
# creating array using list
aray_list1 = np.array(list1)
print(arr_list1)
```

```
[1 2 3 4]
```

In [154…
```
# use .ndim to check dimention of array
print(arr_list1.ndim)
```

```
1
```

In [155…
```
print(type(arr_list1))
```

```
<class 'numpy.ndarray'>
```

In [156…
```
# creating 2-D Array using 2 lists

list_1=[1,2,3]
list_2= [4,5,6]
arr_list1_2 = np.array([list_1 , list_2])
print(arr_list1_2)
```

```
[[1 2 3]
 [4 5 6]]
```

In [157…
```
# use .ndim to check dimention of array

print(arr_list1_2.ndim)
```

```
2
```

In [158…
```
# creating 3-D Array using 3 lists

list_1=[1,2,3]
list_2= [4,5,6]
list_3=[7,8,9]
arr_list_1_2_3 = np.array([[list_1 , list_2 , list_3]])
print(arr_list_1_2_3)
```

```
[[[1 2 3]
  [4 5 6]
  [7 8 9]]]
```

In [159…
```
# use .ndim to check dimention of array

print(arr_list_1_2_3.ndim)
```

```
3
```

2. numpy array attributes

In [162…

```
arr1 =np.array([[1,2,3],[4,5,6]])
print(arr1)
```

```
[[1 2 3]
 [4 5 6]]
```

In [163…

```
# Attributes of Array (properties of array)
print("shape", arr1.shape)
print("size", arr1.size)
print("dType", arr1.dtype)
print("n_dim", arr1.ndim)
```

```
shape (2, 3)
size 6
dType int64
n_dim 2
```

3. Array initialization methods

In [165…

```
# zeros array : all elements are 0

zero_arr = np.zeros((2,3))
print(zero_arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

In [166…

```
# Ones array : all elements are 1

ones_arr = np.ones((2,3))
print(ones_arr)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

In [167…

```
# Full array : all elements are same based on given value

full_arr = np.full((3,2),9)
print(full_arr)
```

```
[[9 9]
 [9 9]
 [9 9]]
```

In [168…

```
full_arr1 = np.full((3,2),7)
print(full_arr1)
```

```
[[7 7]
 [7 7]
 [7 7]]
```

Identity Matrix : diagonal elements are ones and all other elements are zeros

```
An identity matrix is a square matrix with:
# 1s on the main diagonal
# 0s everywhere else
# row = column
```

In [16]:
```
id_arr = np.eye(3)
print(id_arr)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [170…
```
# Empty array

print(np.empty(2))
```

```
[1.5e-323 1.5e-323]
```

In [171…
```
# Evenly Spaced array : elements are evenly spaced based on step value

print(np.arange(2,10,2))
print(np.arange(1,9,1))
```

```
[2 4 6 8]
[1 2 3 4 5 6 7 8]
```

In [172…
```
# Specific numbers of equally spaced value between a range : elements are equall
print(np.linspace(1,10,4))
```

```
[ 1.  4.  7. 10.]
```

In [173…
```
# random values array -------- float

r_arr = np.random.rand(2,3)   # (start, end, (matrix size))
print(r_arr)
```

```
[[0.00588951 0.05628713 0.85070904]
 [0.88604415 0.43138389 0.56790425]]
```

In [174…
```
# random values array -------- int

rint_arr = np.random.randint(1,10,(3,3))   # (start, end, (matrix size))
print(rint_arr)
```

```
[[5 7 8]
 [3 1 5]
 [4 5 2]]
```

4. Array Indexing and Slicing

In [176…
```
a= np.array([1,3,5,7,9])
print(a)
```

```
[1 3 5 7 9]
```

In [178…
```
print(a[0])    # first element of array
```

```
1
```

In [179…
```
print(a[4])   # fifth element of array
```

```
9
```

In [180…
```
print(a[-1])    # last element of array
```

```
9
```

In [181…

```
a= np.array([1,3,5,7,9])
print(a)
# slicing:
# array[start:stop:step]
# stop - index, that is not included in output
```

```
[1 3 5 7 9]
```

In [184…

```
# first 3 elements
print(a[0:3])
```

```
[1 3 5]
```

In [185…

```
# middle 3 elements
print(a[1:4])
```

```
[3 5 7]
```

In [186…

```
# last 3 elements
print(a[-3:])
```

```
[5 7 9]
```

In [187…

```
print(a[1:4]) # Middle 3 elements
```

```
[3 5 7]
```

In [188…

```
# middle 3 elements with steps
print(a[0:4:1])
print(a[0:4:2])
```

```
[1 3 5 7]
[1 5]
```

In [142…

```
# Indexing  and Slicing in 2-D array
```

In [189…

```
# Indexing on 2-D Array
arr2= np.array([[1,2,3],[4,5,6]])
print(arr2)

# 2D Array: [rows, columns]                    # 2D Array takes 2 values - rows a
# 3D Array: [layers/height, rows, columns]     # 3D Array takes 3 values - height
```

```
[[1 2 3]
 [4 5 6]]
```

In [192…

```
print(arr2[0]) # first row
print(arr2[1]) # second row
```

```
[1 2 3]
[4 5 6]
```

In [193…

```
# Slicing on 2D array
print(arr2[0][0]) # element of first row and first column
print(arr2[0][1]) # element of first row and second column
print(arr2[1][2]) # element of second row and third column
```

```
1
2
6
```

In [194…

```
print(arr2[1]) # specific element - 1D
```

```
print(arr2[1:]) # slicing part - 2D
```

```
[4 5 6]
[[4 5 6]]
```

In [195…

```
print(arr2)

# 2D Array: [rows, columns]  |  eg: arr2[0][1]     # 2D Array takes 2 values - r
```

```
[[1 2 3]
 [4 5 6]]
```

In [196…

```
print(arr2[:, 0])       # First column of an array
print(arr2[:, 1])       # Second column of an array
print(arr2[:, 2])       # Third column of an array
```

```
[1 4]
[2 5]
[3 6]
```

In [197…

```
arr3 = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9]])
print(arr3)

# 3D Array: [layers/height, rows, columns]        # 3D Array takes 3 values - he
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [198…

```
print(arr3[0])
print(arr3[1])
print(arr3[2])
```

```
[1 2 3]
[4 5 6]
[7 8 9]
```

In [199…

```
print(arr3[0][2])
print(arr3[2][2])
```

```
3
9
```

In [200…

```
print(arr3[:, 0]) # 1st col value
print(arr3[:, 1]) # 2nd col value
print(arr3[:, 2]) # 3rd col value
```

```
[1 4 7]
[2 5 8]
[3 6 9]
```

In [201…

```
print(arr3[::, 1:2])  #slicing on columns
print(arr3[2::, 1:2]) #slicing on rows and columns
```

```
[[2]
 [5]
 [8]]
[[8]]
```

5. Array Reshaping and Flattening

Reshape Array: changing its dimensions (e.g., rows and columns) without altering the actual data

Flatten Array: transforming a multi-dimensional array into a single-dimensional array

In [8]:
```
arr1= np.array([1,2,3,4,5,6])    # shape = (1,6) - 1 row and 6 columns
print(arr1)
```

```
[1 2 3 4 5 6]
```

In [9]:
```
# Reshaping
reshaped = arr1.reshape((6,1))    # shape = (6,1) - 6 rows and 1 column
print(reshaped)
```

```
[[1]
 [2]
 [3]
 [4]
 [5]
 [6]]
```

In [10]:
```
reshaped2 = arr1.reshape((2,3))     # re-shaped array from (1,6) to (2,3)
print(reshaped2)
```

```
[[1 2 3]
 [4 5 6]]
```

Both convert an array into 1D, but:

ravel() → returns a view (shares memory)

flatten() → returns a copy (new memory)

In [11]:
```
#  ravel              convert 1D array
ravel = reshaped2.ravel()
print(ravel)
```

```
[1 2 3 4 5 6]
```

In [216…
```
# Flattening
print(reshaped2.flatten())     # flattened array from higher (2D) dimensional (2
```

```
[1 2 3 4 5 6]
```

6. Array Stacking and Splitting

Array Stacking: combining multiple arrays along a new axis, resulting in an array with a higher dimension than the input arrays

Array Splitting: Divides an array into multiple smaller sub-arrays

In [37]:
```
# STACKING

a= np.array([1,2,3])
b=np.array([4,5,6])

print(np.vstack((a,b)))  # vertical Stacking - row wise
print()
print(np.hstack((a,b)))  # Horizontal Stacking - column wise
```

```
[[1 2 3]
 [4 5 6]]

[1 2 3 4 5 6]
```

In [12]:
```
c= np.array([[1,2,3],[4,5,6]])
print(c)
```

```
[[1 2 3]
 [4 5 6]]
```

In [ ]:
```
# SPLITTING

split= np.hsplit(c,3)  # Horizontal split

for s in split:
    print(s)
```

In [15]:
```
split = np.vsplit(c,2)  # verticall split

for s in split:
    print(s)
```

```
[[1 2 3]]
[[4 5 6]]
```

7. Mathematical Operatoins on Array

In [13]:
```
a= np.array([10,20,40,-30])
print(a)
```

```
[ 10  20  40 -30]
```

In [14]:
```
print(a+10) # add
print(a-30) # subs
print(a*2) # multiply
print(a/10) # divide
```

```
[ 20  30  50 -20]
[-20 -10  10 -60]
[ 20  40  80 -60]
[ 1.  2.  4. -3.]
```

In [15]:
```
b= np.array([1,4,9])
print(b)
```

```
[1 4 9]
```

In [16]:
```
print(np.square(b))

print(np.sqrt(b))
```

```
[ 1 16 81]
[1. 2. 3.]
```

8. Mathematical Operatoins on Multiple Array

In [22]:
```
a = np.array([1,2,3])
b = np.array([4,5,6])
print(a)
print(b)
```

```
[1 2 3]
[4 5 6]
```

In [23]:
```
print(np.add(a,b))          # add two arrays
print(np.subtract(a,b))     # subtract two arrays
print(np.multiply(a,b))     # multiply two arrays
print(np.divide(a,b))       # divide two arrays
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

In [24]:
```
# Dot product: computes the inner product of the two vectors and sum the results

print(np.dot(a,b)) # dot product
```

```
32
```

In [25]:
```
print(a.T) # Transpose: Returns an array with axes transposed
```

```
[1 2 3]
```

In [42]:
```
t = np.array([[1,2,3],
              [4,5,6]])

print(t)    # initial shape: 2,3
print()
print(t.T)  # shape after transpose: 3,2
```

```
[[1 2 3]
 [4 5 6]]

[[1 4]
 [2 5]
 [3 6]]
```

9. Statistical Functions

In [4]:
```
a= np.array([[1,2,3],[4,5,6]])
print(a)
```

```
[[1 2 3]
 [4 5 6]]
```

In [33]:
```
print(np.sum(a))   # sum of all elements of an array
```

```
21
```

In [32]:
```
print(np.mean(a))
print(np.median(a))
print(np.std(a))
print(np.min(a))
print(np.max(a))
```

```
3.5
3.5
1.707825127659933
1
6
```

10. Array Comparison

In [34]:
```
a=np.array([1,5,3])
b=np.array([4,5,6])
print(a)
print(b)
```

```
[1 5 3]
[4 5 6]
```

In [35]:
```
print(a == b)      # comparing each elements of an array
print(np.array_equal(a,b))   # comparing complete array
```

```
[False  True False]
False
```

11. Broadcasting

Broadcasting is a mechanism that allows arithmetic operations to be performed on arrays of different shapes and sizes, it eliminates the need for explicit loops or reshaping operations, making code more concise, readable and efficient.

Broadcasting Rules:

```
# Compare shapes from right to left.
# Dimensions must match or be 1.
# If one array has fewer dimensions, pad its shape on the left
with 1s.
# Any dimension equal to 1 can stretch to match the other array.
```

In [40]:
```
a=np.array([1,2,3,4])
b= np.array([5])
print(a+b)         # adding 1D with 0ne-element array to 1D with multi-element a
```

```
[6 7 8 9]
```

In [39]:
```
c=np.array([[1,2,3],[4,5,6]])
d = np.array([10,20,30])
print(c+d)         # adding 2D with 6-element array (2,3) to 1D with 6-element a
```

```
[[11 22 33]
 [14 25 36]]
```

12. Handling with nan & inf

(Dealing with Missing Values)

nan = none

inf = infinite

In [41]:
```
data1 =  np.array([1,2, np.nan, 4, np.inf])
print(data1)
```

```
[ 1.  2. nan  4. inf]
```

In [42]:
```
print(np.isnan(data1))  # check null value and returns boolean value
```

```
[False False  True False False]
```

In [43]:
```
print(np.nan_to_num(data1))    # replace non-finite values within an array with
```

```
[1.00000000e+000 2.00000000e+000 0.00000000e+000 4.00000000e+000
 1.79769313e+308]
```

13. Views vs Copies

In [17]:
```
arr= np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

In [30]:
```
# VIEWS

view = arr[1:3]
print(view)
view
```

```
[200   3]
```

Out[30]:
```
array([200,   3])
```

In [31]:
```
view[0] = 200
print(view)
```

```
[200   3]
```

In [32]:
```
print(arr)
```

```
[  1 200   3   4   5]
```

In [33]:
```
# COPY

copy = arr[1:3].copy()
print(copy)
```

```
[200   3]
```

In [34]:
```
copy[0] = 2
print(copy)
```

```
[2 3]
```

In [35]:
```
print(arr)
```

```
[  1 200   3   4   5]
```

14. Transpose of a Matrix

In [38]:
```
arr= np.array([[1,2],[3,4]])
print(arr)
```

```
[[1 2]
 [3 4]]
```

In [40]:
```
# Transpose
print(arr.transpose())
```

```
[[1 3]
 [2 4]]
```

15. Conditional-Based Operations on Array

In [45]:
```
arr = np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

In [48]:
```
result1 = np.where(arr <2,"low","high")   # WHERE - condition
print(result1)
```

```
['low' 'high' 'high' 'high' 'high']
```

In [51]:
```
# ArgWhere : Row-Column positions for 2nd arrays

arr= np.array([10,25,30,50])
print(arr)
```

```
[10 25 30 50]
```

In [54]:
```
print(np.argwhere(arr>20))    # argwhere - condition
```

```
[[1]
 [2]
 [3]]
```

In [55]:
```
mask = np.logical_and(arr>15, arr< 35)       # logical_and  - condition
print(mask)
```

```
[False  True  True False]
```

16. Save & Load Arrays

In [4]:
```
# SAVE

arr = np.array([1,0,0,8])
print(arr)
np.save('my_array.npy',arr)
```

```
[1 0 0 8]
```

In [5]:
```
# LOAD
loaded_arr =np.load('my_array.npy')
print(loaded_arr)
```

```
[1 0 0 8]
```